

Functional Coverage

SyoSil, 2026

Contents

- Coverage Intro
- Structural Coverage
- Functional Coverage
- Functional Coverage and UVM Test Bench
- Functional Coverage and SystemVerilog
- Covergroup
- Coverpoint
- Bins
- Cross
- Sampling
- Options & Methods
- Wildcard
- What to Watch Out For?

Coverage Intro

Novel idea: Constrained Random Verification (CRV)

- AKA Coverage Driven Verification - CDV
- When the computer have to write the tests then how to produce:
 - **Produce Stimuli?**
 - **Check Correctness?**
 - **Check Completeness?**
 - If **stimuli** is random then **Correctness** and **Completeness** cannot be computed per test
- Thus, 1D approach has to be split!
 - Into individual dimensions (3D) as the computer will write test by generating **stimuli** not knowing the **completeness** and **correctness** criterias.

To produce a machine written test, then the computer needs to be able to independently:

- Produce **stimuli**
- Check/report **correctness**
- Check/report **completeness**

Check/Report Completeness → Coverage!

- Constrained random verification (CRV) means looser coupling of testcases and verification completeness
- We need explicit measurement (Coverage) for verification completeness
- Verification completeness == What parts of the functional state space has been covered!

Types of Coverage

- **Structural Coverage**

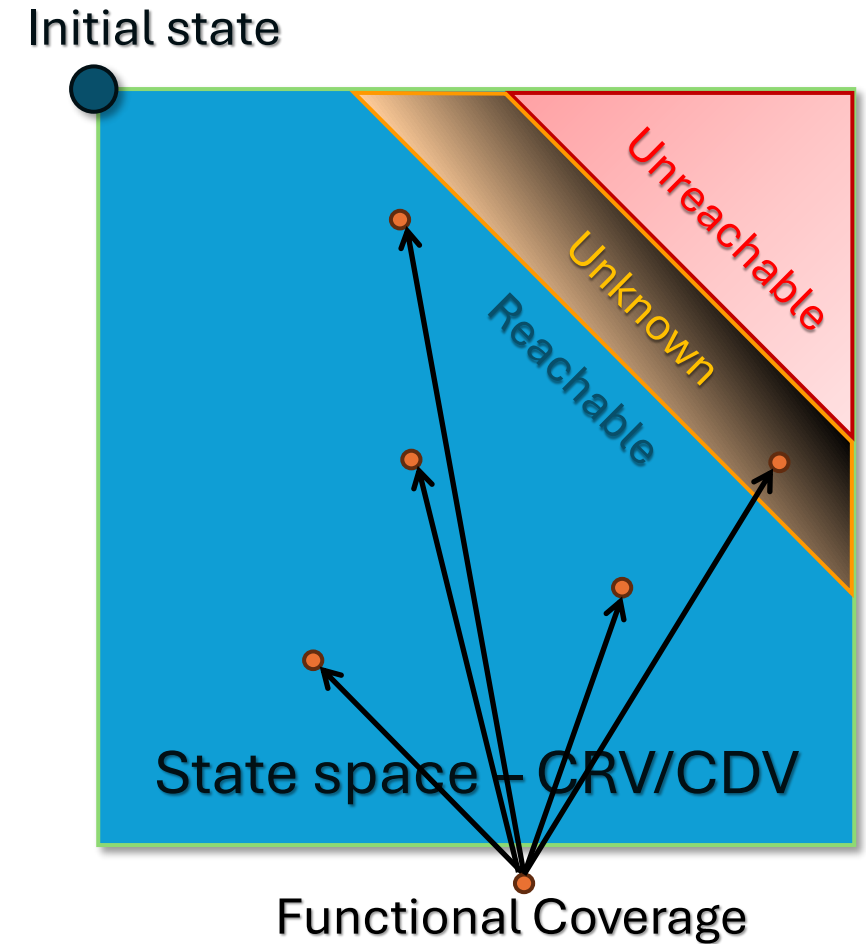
- Relates to the source code of the RTL
- Can be extracted automatically while simulating the RTL
- No implementation required

- **Functional Coverage**

- Which parts of the functional state space have been covered
- Requires implementation of a functional coverage model

- **Verification completeness**

- Structural+Functional coverage complete



NOTE! Covering all of the structural coverage does NOT ensure full functional coverage!

Structural Coverage

Structural Coverage

- HDL Structural Coverage has arisen from analysis of SW projects
 - Specifically, the statistical analysis derived from instrumentation/analysis of executing code
- Pros:
 - White-box view of design from within
 - Tool generated, so less burden on designers
 - Finds areas of a program not exercised
 - Targets additional test cases to increase coverage
 - Provides a measure of code coverage and indirectly of quality
 - May identify redundant test cases that do not increase coverage
- Cons:
 - Derived from work on linear languages (C, C++)
 - What about concurrency of HDL's?
- It focuses on ensuring that various structural elements of the design have been executed.
- Common types of structural coverage:
 - Line/Statement
 - Block
 - Branch/Decision
 - Conditional/Expression
 - Toggle
 - FSM (State/Transition)

Structural Coverage Example

What did simulation execute in this RTL?

RTL UNDER TEST

```
always @(posedge clk) begin
  if (rst === 1'b0) begin
    c <= 0;
  end else begin
    c <= a + b;
  end
end
```

Two control outcomes
Reset branch | Add branch

STRUCTURAL METRICS

Statement/Line	Did each executable statement run? <code>c <= 0; and c <= a + b;</code>
Branch	Did the if take both outcomes? <code>rst === 1'b0 true / false</code>
Condition	Which Boolean result did rst produce? <code>0 / 1 / X / Z</code> can matter
Toggle	Did signal bits change 0↔1? <code>clk, rst, a, b, c</code> (tool dependent)

Coverage shows exercised structure. It does not prove that the full state space is traversed.

For complete toggle coverage, define the tool’s rules for clock and reset signals. Functional goals need separate checks and coverage.

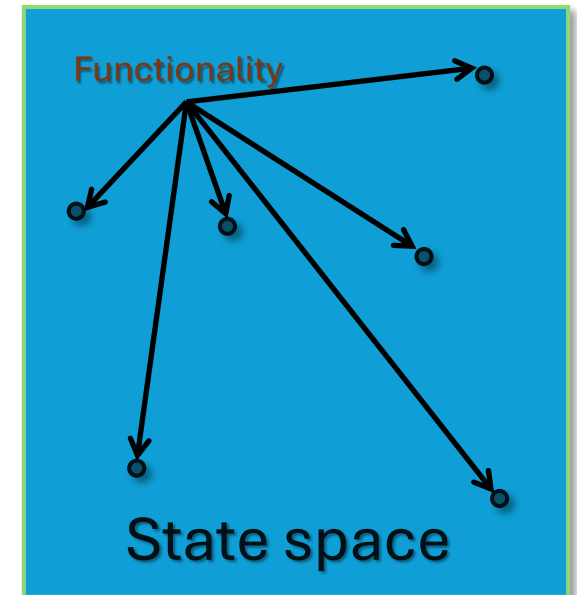
Structural Coverage for cocoTB/PyUVM TBs

- Not available in our flow (Yet!)
 - Icarus
 - No native support for structural coverage
 - 3rd party tool: Covered (<https://covered.sourceforge.net/>)
 - Not maintained anymore
 - Verilator
 - Maintained structural coverage
 - Maybe we should do a simulator switch?

Functional Coverage

Functional State Space Example

- Device
 - Adder with 2, 8bit, ports, a and b
 - One 9 bit output port
 - No internal state
 - Number of states in functional state space:
 - Input (for each clk):
 - rst: 2^1
 - a: 2^8
 - b: 2^8
 - Total: $2 \cdot 256 \cdot 256 = 131072$
 - Output: $2^9 = 512$



RTL UNDER TEST

```
module adder(  
    input  logic clk,  
    input  logic rst,  
           logic [7:0] a,  
           logic [7:0] b,  
    output logic [8:0] c);  
  
    always @(posedge clk) begin  
        if(rst === 1'b0) begin  
            c <= 0;  
        end else begin  
            c <= a+b;  
        end  
    end  
end  
endmodule
```

Functional Coverage

- Functional Coverage tries to capture interesting parts of the functional state space
- Example of interesting parts
 - Have all typical operational modes been tested
 - All error conditions? All corner cases?
- Pros:
 - Black-box, external view
 - Targets the stimulus/testbench more than the actual code
 - Adaptable to more efficient transaction-level analysis, not just signal level
 - Identifies overlap/redundancy in test cases
 - Fits excellently with Assertion Based Verification & Constrained Random
 - Fits with Transaction-based verification
- Cons:
 - Some effort initially to implement
 - Poor implementation will yield poor results

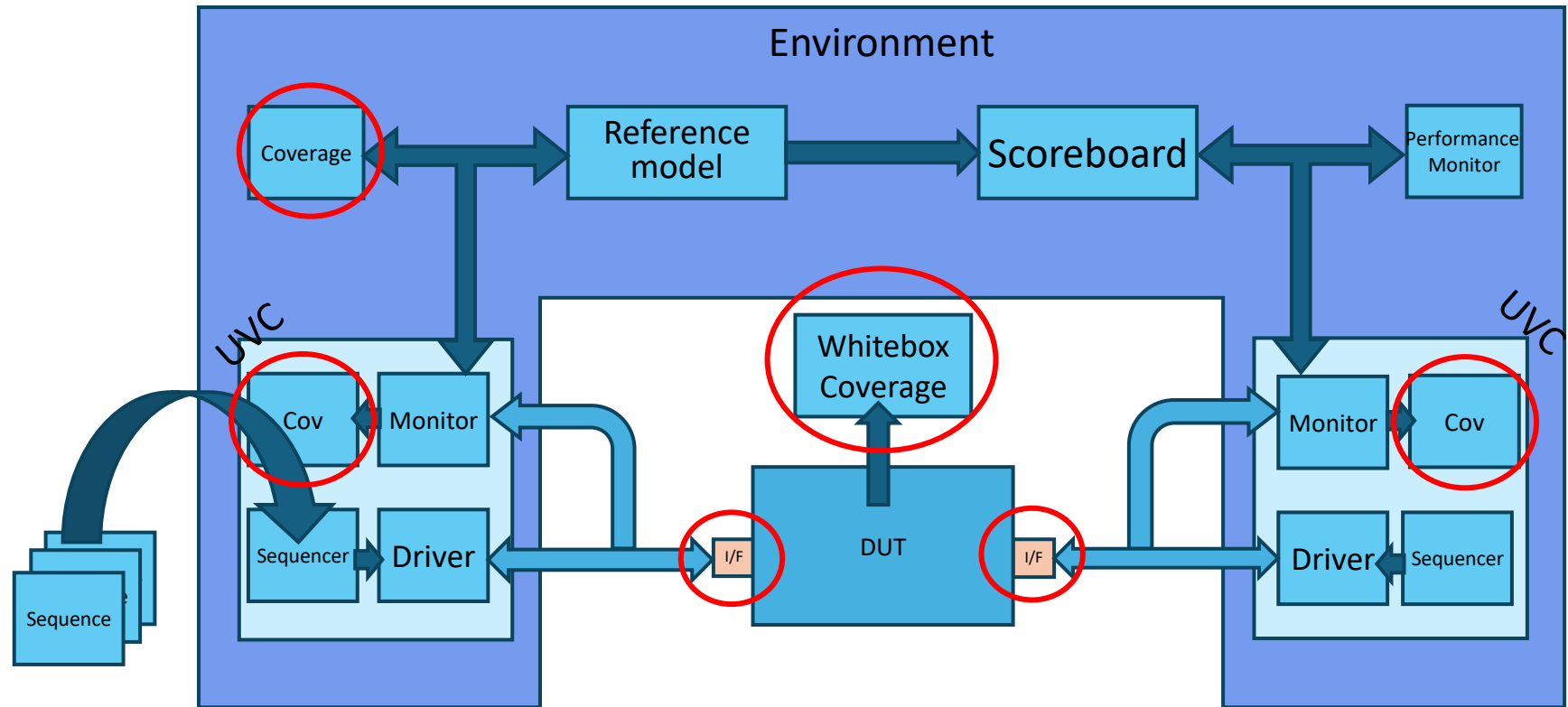
Why Functional Coverage?

- Structural coverage is simply not sufficient – period!
- Imagine
 - DUT with two mode registers (with 3 and 5 modes each)
 - Structural coverage will have a hard time showing which permutations of the 15 combined modes the DUT can be in
 - Only if the DUT implementation somehow branches out specifically on this
 - Thus, structural coverage just tells you if you have written the two registers or not
- Functional coverage lets you express that you need to ensure that the DUT has been observed to be in all 15 permutations
- Same for datapath. You need to ensure that certain patterns have been observed.

Functional Coverage and UVM Test Bench

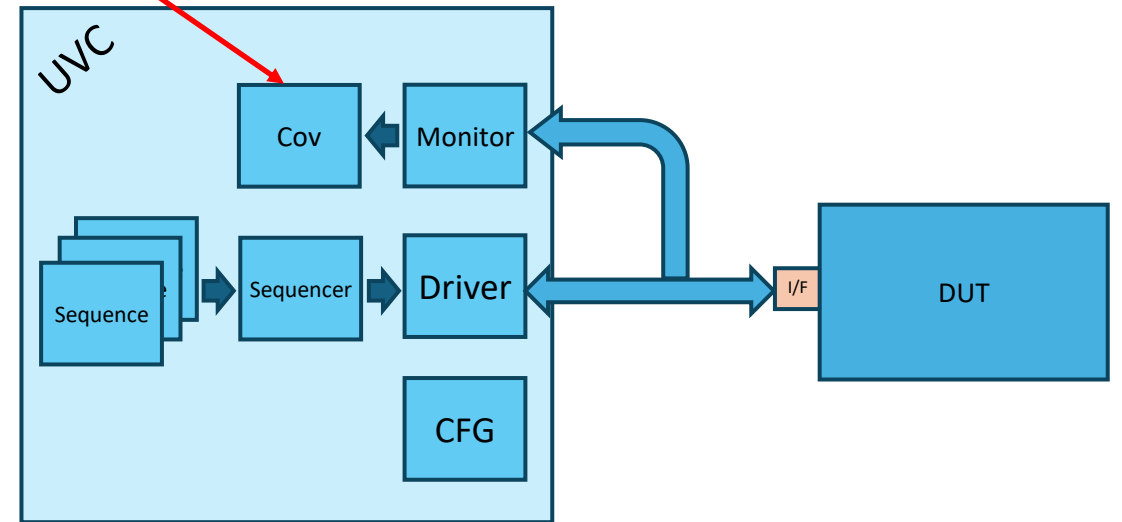
Where to place functional coverage?

- Interfaces
 - Protocol/Pin coverage
- UVC cov object
 - Protocol coverage
 - Transaction coverage
- Testbench
 - Single global coverage collector
- Whitebox coverage
 - Using bind



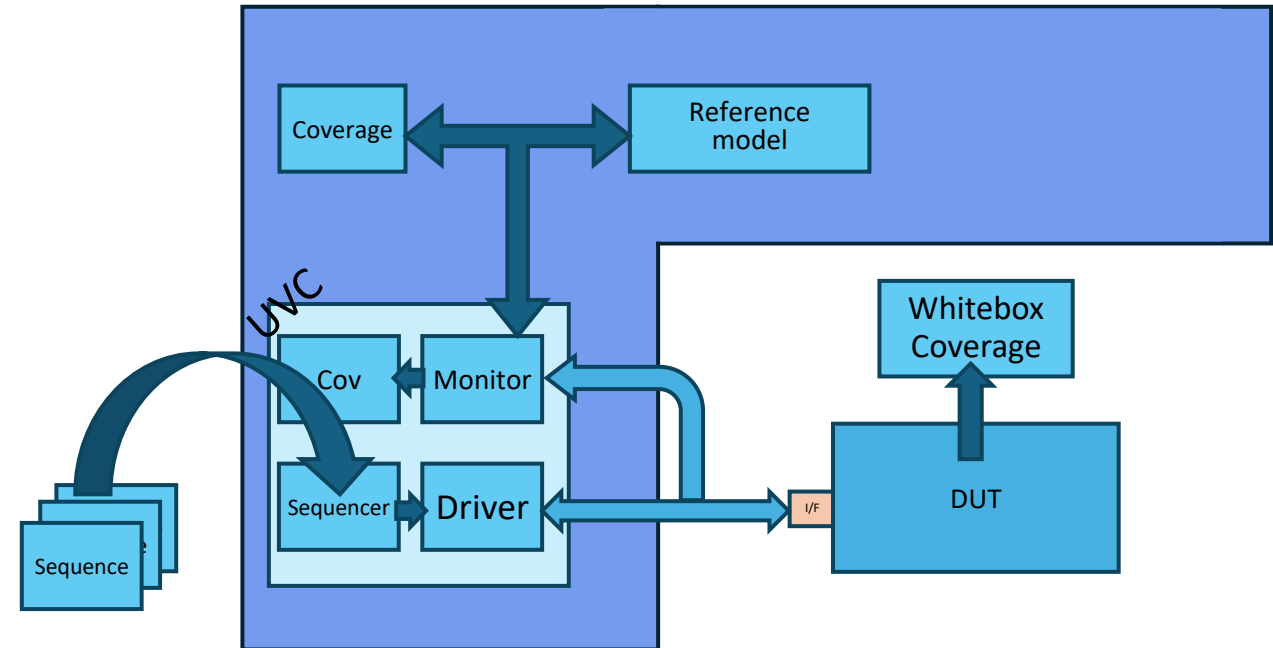
UVC Coverage Collector

- Extended from `uvm_subscriber`
 - Specializations should override the `write()` function.
- Subscribing to monitor observed sequence items
- Protocol centric coverage model
 - SV functional coverage
 - Examples:
 - Access types
 - Address map
 - Transaction Ids



UVM ENV Coverage Collector

- Single global coverage collector
 - → Why? More robust over TB lifetime
 - All information is available
 - Captures DUT specific functional coverage
 - Modes, Corner cases, ...
- Connected to
 - All UVCs
 - Virtual Sequencer
 - Allows virtual sequences to provide information and/or trigger sampling



NOTE: Many teaches to implement a distributed coverage model but that can potentially break over time as it might be difficult to express all parts of the state space to be observed since not all information is available

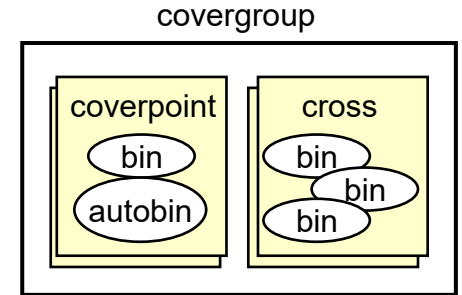
Functional Coverage with PyUVM (PyVSC)

Functional Coverage for PyUVM

- PyVCS supports implementation of functional coverage
 - <https://pyvsc.readthedocs.io/en/latest/index.html>
- You can use it to implement a functional coverage model
- It is a basic port of the SystemVerilog functional coverage
 - “with” clause not supported
 - Sampling at clock events not supported
 - Binsof and intersect not supported in crosses
- Main features
 - Covergroup
 - Coverpoints
 - Bins
 - Crosses

<https://pyvsc.readthedocs.io/en/latest/coverage.html>

PyVSC Coverage Terminology



Keywords

- **covergroup**
 - Toplevel container for coverpoints and crosses
 - Composed of a number of sub-elements including the following...
- **coverpoint**
 - Describes a particular type of coverage event, how it will be counted as well as one or more bins to organize the count
- **bins**
 - A coverage event counting mechanism, automatically or user-defined
- **cross**
 - Defines a new coverage event by “combining” two or more existing coverpoints or variables
- Guard (**iff** <expr>)
 - Optional condition that disables coverage at the covergroup, coverpoint, cross or bins level

Covergroup

Covergroups

- Contain a number of values to be sampled → **coverpoints**
- **Coverpoints** sample an expression and determine if it falls into a number of bins
 - Bins can be automatically generated or manually specified
- All **coverpoints** in a **covergroup** are sampled at the same time
- **Crosses** can be specified between 2 or more **coverpoints** in the same **covergroup**
- **Covergroups** are created by classes

Declaring a **covergroup**

- **covergroup**
 - In PyVSC a covergroup is defined by implementing sequential code which builds the covergroup
 - In SystemVerilog it is declarative
 - Typically, the covergroups are wrapped by a class

Covergroup

```
@vsc.covergroup
class my_covergroup(object):
    def __init__(self):
        pass

my_cg_1 = my_covergroup()
my_cg_2 = my_covergroup()
```

*instances of coverage model
(multi-instantiation is OK)*

Coverpoint

Declaring a **coverpoint**

- Each **coverpoint** samples one expression
 - Into a number of bins which are either auto or user-defined
- **Coverpoint** defines a number of bins a sampled value may fall into
- A **coverpoint** may define ignore bins
- A **coverpoint** may also defined illegal bins

Coverpoint

```
@vsc.covergroup
class my_covergroup(object):
    def __init__(self):
        self.cp1 = vsc.coverpoint(self.a, bins={
            "b0" : vsc.bin(1, 2, 4),
            "b1" : vsc.bin(8, [12,15])
        })

my_cg_1 = my_covergroup()
my_cg_2 = my_covergroup()
```

Sampling and Sampling Conditions

Covergroup Sampling

- In SystemVerilog, Covergroups can be sampled in 3 different ways
 - Clock event
 - Sequence/Method Sampling
 - Procedural Sampling
 - Explicitly calling **sample()** function
- For PyVSC only procedural sampling can be done
 - Clock event can be emulated as one can just sample every clock procedurally 😊
- In UVM common usage is procedural sampling
 - This matches transaction modelling and allows passing of arguments

Procedural Coverage Sampling

- Sampling can be performed by calling the built-in `sample()` method

```
@vsc.covergroup
class my_covergroup(object):
    def __init__(self):
        self.cp1 = vsc.coverpoint(self.a, bins={
            "b0" : vsc.bin(1, 2, 4),
            "b1" : vsc.bin(8, [12,15])
        })

cg = my_covergroup()
cg.sample(1)
cg.sample(12)
```

Custom Sampling Method

- The sampling method can be customized by using the **with_sample** method
- The **with_sample** method declares class members with the same name and type as the key/value pairs in the dict
- The **with_sample** method should be called early in the `__init__` method body to ensure that the sample parameters are present when referenced in coverpoints

```
@vsc.covergroup
class my_covergroup(object):

    def __init__(self):
        self.with_sample(
            a=vsc.bit_t(4)
        )
        self.cp1 = vsc.coverpoint(self.a, bins={
            "b0" : vsc.bin(1, 2, 4),
            "b1" : vsc.bin(8, [12,15])
        })

cg = my_covergroup()
cg.sample(1)
cg.sample(12)
```

Bins

Individual Bins and Bin Arrays

```
@vsc.covergroup
class my_covergroup(object):
    def __init__(self, a : callable):

        self.cp1 = vsc.coverpoint(a, bins={
            "b0" : vsc.bin(1, 2, 4),
            "b1" : vsc.bin(8, [12,15])
        })
```

- Individual Bins

- **b0** bin contains the values 1, 2 and 4
- **b1** bin contains the values 8 and the range 12 to 15

```
@vsc.covergroup
class my_covergroup(object):

    def __init__(self, a : callable):

        self.cp1 = vsc.coverpoint(a, bins={
            "b0" : vsc.bin_array([], 1, 2, 4),
            "b1" : vsc.bin_array([4], [8,15])
        })
```

- Bin Arrays

- **b0** bin contains 3 bins, one for each of the values 1, 2 and 4
- **b1** bin contains 4 bins covering the range 8 to 15 (8..9, 10..11, 12..13, 14..15)

Auto **bins**

- If no user-defined **bins** are declared for a coverpoint, **bins** are automatically created.

```
@vsc.covergroup
class my_covergroup(object):
    def __init__(self):
        self.options.auto_bin_max = 16
        self.with_sample(a=vsc.bit_t(4))

        self.cp1 = vsc.coverpoint(self.a)

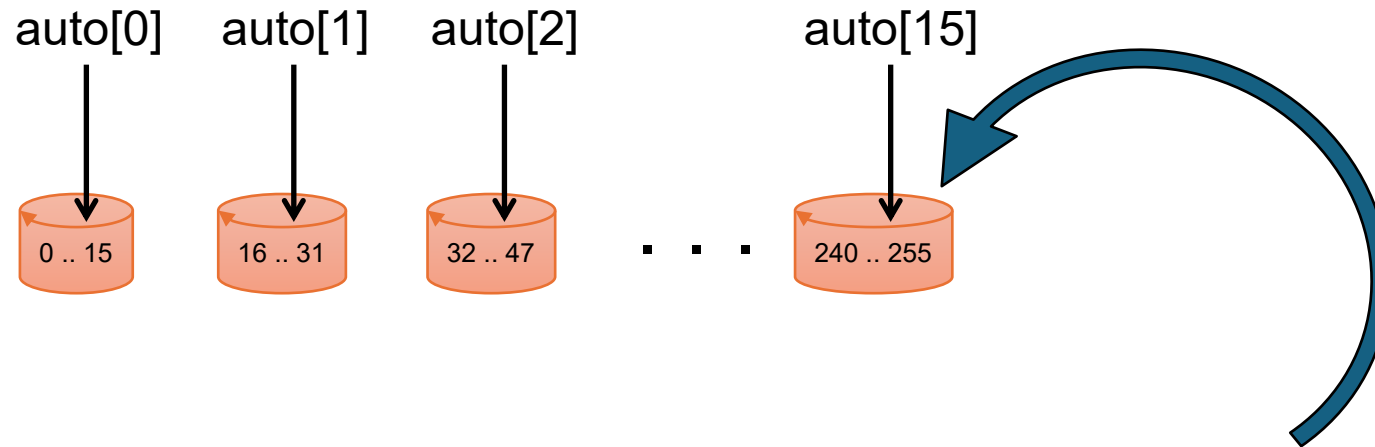
my_cg_1 = my_covergroup()
```

To prevent auto-bin explosion, the number of auto **bins** created for a coverpoint is defined as the lower of:

- 2^N (The number of **bins** required for full coverage of **coverpoint**)
- **auto_bin_max**, (a predefined coverage option (default 64))

Assigning values to auto-bins

- If the number of potential values for an expression exceeds the number of bins available
 - Bins are “spread out” evenly to cover ranges of values



- If the number of bins does not divide evenly into the range of values, any remaining values are grouped into the range of the final bin

Wildcard Bins

```
@vsc.covergroup
class cg(object):
    def __init__(self):
        self.with_sample(
            dict(a=vsc.bit_t(8)))

        self.cp_a = vsc.coverpoint(self.a, bins=dict(
            a=vsc.wildcard_bin("0x8x")))
```

String Example: wildcard bin that matches any value 0x80..0x8F

```
@vsc.covergroup
class cg(object):
    def __init__(self):
        self.with_sample(
            dict(a=vsc.bit_t(8)))

        self.cp_a = vsc.coverpoint(self.a, bins=dict(
            a=vsc.wildcard_bin((0x80,0xF0))
        ))
```

Value/mask: wildcard bin that matches any value 0x80..0x8F

- A wildcard specification may be used to specify the values within single bins.
 - The checked value may either be specified as a string that contains wildcard characters ('x', '?') or may be specified as a tuple of (value, mask).

Wildcard Bins Array

- A wildcard specification may also be used to specify arrays of bins
 - In this case, the wildcard characters specify a location where all possibilities must be expanded.
- The example below creates 16 bins for the values 0x80..0x8F

```
@vsc.covergroup
class cg(object):
    def __init__(self):
        self.with_sample(
            dict(a=vsc.bit_t(8)))

        self.cp_a = vsc.coverpoint(self.a, bins=dict(
            a=vsc.wildcard_bin_array([], "0x8x")
        ))
```

Exclude Bins (Ignore and Illegal Bins)

- Ignore and illegal bins may be specified on **coverpoints** in addition to the other bins described above.
 - An ignore or illegal bin trims values from other bins if it intersects values within those bins.
 - Please note that, as in SystemVerilog, bins are partitioned after ignore and illegal bin values are removed from regular bins.

Exclude Bins (Ignore and Illegal Bins)

```
@vsc.covergroup
class val_cg(object):
    def __init__(self):
        self.with_sample(dict(
            a=vsc.uint8_t()
        ))

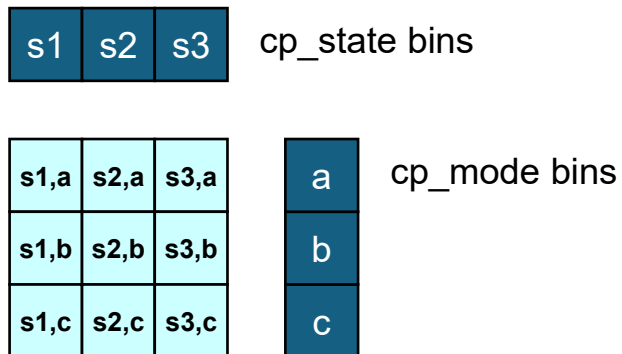
        self.cp_val = vsc.coverpoint(self.a, bins=dict(
            rng_1=vsc.bin_array([4], [1,3], [4,6], [7,9], [10,12])
        ),
            ignore_bins=dict(
                ignored_value=vsc.bin(4)
            ))
```

- A Bin Array of four bins and an ignored value of 4
- In the absence of ignore bins, the 12 values to be partitioned as bins of three (1..3, 4..6, 7..9, 10..12)
- Because bins are partitioned after excluded bins have been applied, the bins in the example above are: 1..3, 5..7, 8..10, 11..12

Cross

Cross coverage

- A coverpoint typically defines coverage for a single variable or expression.
 - Answers questions like:
 - Have I entered every state of the FSM?
 - Have I tested all operating modes?
- Sometimes we need to study combinations of two or more coverpoints simultaneously.
 - Answers questions above PLUS :
 - Have I entered every state WHILE being in each mode?



```
@vsc.covergroup
class my_covergroup(object):
    def __init__(self, a : callable):
        self.cp_state = vsc.coverpoint(..., bins={
            "s1" : vsc.bin(0),
            "s2" : vsc.bin(1),
            "s3" : vsc.bin(2),
        })

        self.cp_mode = vsc.coverpoint(..., bins={
            "a" : vsc.bin(0),
            "b" : vsc.bin(1),
            "c" : vsc.bin(2),
        })
```

Cross Coverage

- Cross coverage is coverage between two or more coverpoints
- Leads to n-dimensional matrix – combinatorial explosion of bins to be covered

```
@vsc.covergroup
class my_covergroup(object):
    def __init__(self, a : callable):
        self.cp_state = ...
        self.cp_mode = ...

        self.cr_state_mode = vsc.cross([self.cp_state, self.cp_mode])
```

Options & Methods

Coverage Options for Covergroup

- PyVSC covergroups contain an *options* attribute and a *type_options* attribute
 - Used to configure covergroup options.
 - Options may only be changed within the `__init__` method.

Option Name	Default	Description
name="string"	Unique name	Name of covergroup instance
weight=number	1	The weight of the covergroup instance
goal=number	100	The target goal of the covergroup instance
comment="string"	""	A comment
at_least=number	1	Minimum number of hits
auto_bin_max=number	64	Maximum number of auto generated bins
per_instance=bool	False	When true instance specific coverage is enabled
get_inst_coverage	False	Enables tracking of per-instance coverage with the <code>get_inst_coverage</code> method

Coverage Options for Coverpoint and Crosses

- PyVSC coverpoints (and crosses) contain an *options* attribute and a *type_options* attribute
 - Used to configure coverpoint options.
 - Options may only be changed within the `__init__` method.

Option Name	Default	Description
weight=number	1	The weight of the coverpoint instance
goal=number	100	The target goal of the coverpoint instance
at_least=number	1	Minimum number of hits
auto_bin_max=number	64	Maximum number of auto generated bins

Coverage: Predefined Methods

- A number of built-in methods allow interrogation of the coverage database

`sample()`

`get_coverage()`

`get_inst_coverage()`

- Sample the covergroup

- Cumulative coverage number (all instances) [0-100]

- Cumulative coverage number [0-100]

- To save the coverage database

`vsc.write_coverage_db(filename,...)` - Save coverage DB in UCIS or XML format

- Viewing Coverage with PyUCIS-Viewer

- PyUCIS-Viewer is a very simple graphical viewer for functional coverage data.

NOTE

Coverage is internally calculated as a “real” and reported as a score 0-100

What to Watch out for?

Good Coverage Takes Time!

- Defining a coverage model is not enough
- You need a **good** coverage model, as this is the only metric to the quality of your verification
- It takes time
 - Iterative process
 - Start early! Involve designers, system architects: whoever owns the functional requirements
 - Review, often
- Start big, then eventually reduce

Coverage Culprits & Problems

- **FALSE POSITIVES!!!!!!**
 - Is it valid data?
 - Correct timing?
 - Sampling at the right time is as important as defining a good model
- **ALWAYS** validate with known data
 - Check that the number of bins in e.g. a cross is exactly what you expected?